Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Bu ders dört öğrenci sunumundan oluşur. Sunumlar dersin mikroservis kitabının (Chris Richardson, *Microservices Patterns*) sonraki bölümlerini izler: önceki haftadan kalan CQRS görünümlerinin tasarımı, uçtan uca (end-to-end) testler, güvenli servisler geliştirmek ve Microservice Chassis ile Service Mesh. Sunan kişiler aşağıda "bir öğrenci" diye anılır. Hoca bu derste sunumları yönetti; CQRS'in geçen hafta da işlendiğini hatırlatmak dışında akademik bir ek açıklama yapmadı.

Genel Konular

- Birinci sunum: CQRS görünüm (view) modülünün tasarımı
  - CQRS view modülü (ekrandan, 00:01:40): dışarıdan gelen sorguları uygulayan bir Query API ve event'lere abone olan event handler'lardan oluşur. İkisi de güncelleme ve sorgu işlemlerini bir veri erişim modülü (Data Access) üzerinden view veritabanında yapar; view veritabanına doğrudan erişmezler.
  - Modül tasarımı gereksinimleri (ekrandan, 00:02:20):
    * Veritabanı: kullanılacak veritabanı ve şeması belirlenmeli.
    * Data Access: güncellemeler idempotent olmalı ve eşzamanlı güncellemeler sistemi bozmamalı.
    * Entegrasyon: var olan sisteme yeni bir view modülü eklenecekse ya da şema değişecekse view veritabanını oluşturacak bir mekanizma bulunmalı.
    * Gecikme: istemciler replikasyon gecikmesinden etkilenmemeli.
  - Veritabanı seçimi (ekrandan, 00:03:15): veritabanının sorgu ve güncelleme işlemlerini verimli yapması gerekir; sorgu ve güncellemelerin karakteristiği seçimi belirler. Slayttaki tablo:
    * Birincil anahtarla JSON nesnesi arama: MongoDB, DynamoDB gibi doküman deposu ya da Redis gibi anahtar-değer deposu (örnek: müşteri başına MongoDB dokümanıyla sipariş geçmişi).
    * Sorguyla JSON nesnesi arama: MongoDB ya da DynamoDB (örnek: müşteri görünümü).
    * Metin sorguları: Elasticsearch gibi metin arama motoru (örnek: sipariş başına Elasticsearch dokümanıyla sipariş metin araması).
    * Graf sorguları: Neo4j gibi graf veritabanı (örnek: müşteri, sipariş ve diğer verilerden bir grafla dolandırıcılık tespiti).
    * Geleneksel SQL raporlama/BI: ilişkisel veritabanı (RDBMS) (örnek: standart iş raporları ve analizler).
  - SQL ve NoSQL (ekrandan, 00:04:00): NoSQL veritabanları daha esnek veri modeli, daha iyi performans ve ölçeklenebilirlik sunar, karmaşık işlemlerde geride kalır. CQRS view modülünün işlemleri genelde basit olduğu için NoSQL iyi bir seçimdir. Yine de modern donanımdaki performans, geliştiricilerin SQL tecrübesi, raporlama aracı desteği ve bazı ilişkisel olmayan özellikleri de içerebilmeleri nedeniyle SQL veritabanları da tercih edilebilir.
  - Güncelleme işlemleri (ekrandan, 00:05:20): genelde birincil anahtarla yapılır (siparişler tablosunda istenen sipariş kimliğine sahip satır bulunup güncellenir). Bazı event'ler yabancı anahtar (foreign key) üzerinden güncelleme gerektirir: TeslimatOluşturuldu event'i sipariş kimliğini içerebilir ama TeslimatAlındı event'i içermeyebilir; bu durumda sipariş, teslimat kimliği kullanılarak güncellenir. İlişkisel veritabanları ve MongoDB bunu indekslemeyle verimli yapar; diğer NoSQL veritabanlarında

daha karmaşık işlemler gerekebilir.
  - Veri erişim modülü (DAO ve yardımcı sınıflar): event handler'ların güncelleme işlemlerini ve Query API'nin sorgularını yapar; veritabanı API'si ile yüksek seviyeli koddaki veri tipleri arasında köprü kurar.
  - Eşzamanlı güncellemeler (ekrandan, 00:06:40): aynı kayda aynı anda gelen iki event (örneğin bir sipariş ve bir teslimat event'i) birbirinin güncellemesinin üstüne yazmamalıdır. Önlem: iyimser (optimistic) ya da kötümser (pessimistic) kilitleme.
  - İdempotent güncellemeler (ekrandan, 00:08:30): bir event handler aynı event'le birden çok kez tetiklenebilir (örneğin ağ hatası yüzünden). Aynı event bir kez de birkaç kez de işlense doğru sonuca ulaşılıyorsa handler idempotenttir.
    * Slayttaki örnekte OrderId 123 için "Delivery picked up" ve "Delivery delivered" event'leri tekrarlanır; görünüm PICKED_UP → DELIVERED → PICKED_UP → DELIVERED olur ve üçüncü adımda geçici olarak güncelliğini yitirir ("Temporarily out of date").
    * Bu kabul edilemezse handler tekrarlanan event'leri tespit etmelidir: her event'e eşsiz bir kimlik verilir, handler işlediği kimlikleri kaydeder ve daha önce gelmiş bir kimliği işlemez. Kimlikler sıralı artıyorsa yalnız en büyük kimliği saklamak yeterlidir; ondan küçükler zaten işlenmiştir.
  - Tutarlı veri sağlama (ekrandan, 00:10:00): istemci güncelleme komutundan hemen sonra sorgu yaparsa ve view veritabanı henüz güncellenmemişse istemciye tutarsız veri döner. Çözüm: güncelleme komutunun cevabında güncelleme event'inin kimliğini içeren bir token verilir; istemci sorguyu bu token ile yapar; güncelleme henüz işlenmediyse hata, işlendiyse güncel veri döner.
  - CQRS view ekleme ve güncelleme: yeni sorgu desteği, hata düzeltmesi ya da şema değişikliği nedeniyle uygulamanın ömrü boyunca view eklemek veya güncellemek gerekir. Yeni view için sorgu modülü ve veritabanı hazırlanır, event handler'lar ilgili event'lere abone edilir.
    * Arşivlenmiş event'lerle oluşturma (ekrandan, 00:11:40): bütün event'ler arşivlenir ve yeni view bu arşivden güncel hale getirilir. Sorun: RabbitMQ gibi geleneksel kuyruklar mesajı işlenince siler; Apache Kafka'nın mesaj tutma süresi vardır ama sonsuz değildir. Bu yüzden arşiv için AWS S3 ya da daha ölçeklenebilir Apache Spark gerekir.
    * Artımlı oluşturma: arşivden her seferinde baştan kurmak zamanla daha çok zaman ve kaynak ister. İki adımlı artımlı algoritma: (1) periyodik olarak her aggregate için bir önceki snapshot'tan sonra gelen event'lerle yeni snapshot oluşturulur; (2) view, bu snapshot'lar ve onlardan sonra gelen event'lerle oluşturulur. Checkpoint gibi çalıştığı için maliyet zamanla büyümez.
- İkinci sunum: uçtan uca (end-to-end) testler
  - Tanım: uygulamanın bütün bileşenlerinin birlikte çalıştığını doğrulayan, gerçek kullanım senaryolarını ve kullanıcı etkileşimlerini simüle eden testler.
  - Test piramidinde en üstte yer alır: altta küçük ve hızlı birim testleri, ortada entegrasyon testleri, en üstte sistemin bütününü test eden end-to-end testler. Yavaş, kırılgan ve geliştirmesi zaman alıcı oldukları için az sayıda yazılmalıdır.
  - Zorluklar: yavaşlık (bütün uygulama ve altyapının başlatılması), kırılganlık (çok sayıda hareketli parça; bir hizmet çalışmazsa test başarısız olur, yanlış pozitif sonuçlar), geliştirme ve bakım güçlüğü (bir değişiklik çok sayıda testi etkiler), ölçekleme sorunları (yüzlerce hizmetli sistemlerde yürütme süresi uzar).
  - User journey testi: bir kullanıcının sistemdeki yolculuğuna karşılık gelir. Sipariş oluşturma, revize etme ve iptal etmeyi ayrı ayrı test etmek yerine üçünü yapan tek bir test yazılır; test sayısı ve süresi azalır (ekrandan, 00:20:00).
  - Gherkin: iş paydaşlarının anlayabileceği, Given/When/Then yapısını kullanan alana özgü dil (DSL). End-to-end testler kabul testleri gibi iş odaklı olduğu için üst düzey bir DSL ile yazılması mantıklıdır. Slayttaki örnek (ekrandan, 00:21:00): "Place Revise and Cancel" özelliği; sipariş oluşturulur ve APPROVED olur, toplam 16.33; 2 vegetable samosa eklenince toplam 20.97; iptal edilince CANCELLED.
  - Çalıştırma: bütün uygulama ve altyapı hizmetleri (veritabanı, API'ler) birlikte başlatılır; Gradle Docker Compose eklentisi gibi araçlar gerekli konteynerleri testlerden önce başlatır. FTGO'nun end-to-end test modülünde basit bir uygulama için bütün konteynerleri başlatıp testleri çalıştırmak 4-5 dakika sürebilir.
  - Performans iyileştirme (ekrandan, 00:24:30): user journey testlerine odaklanma, test

süresini optimize etme (paralel çalıştırma), test piramidini dengeli kullanma, otomasyon, hataları erken tespit; araçlar olarak paralel çalıştırma, sanallaştırma (sanal makine, Docker konteyneri) ve CPU/bellek izleme.
  - Özelleştirilmiş end-to-end testler: yalnız belli iş süreçlerini ve kritik user journey'leri hedefleyen, kapsamı daraltılmış testler; daha hızlı, daha az bağımlı, daha az kırılgan.
- Üçüncü sunum: güvenli servisler geliştirmek
  - Bir servisin üretime hazır olması için üç kalite özelliği: güvenlik, yapılandırılabilirlik (ortama özgü bilgiyi dışarıdan alabilme), gözlemlenebilirlik (izleme, günlük kaydı). Sunum güvenliğe odaklanır.
  - Dört güvenlik yönü (ekrandan, 00:30:00): kimlik doğrulama (kullanıcı adı, şifre ya da API anahtarı), yetkilendirme (rol tabanlı güvenlik, erişim kontrol listeleri, ACL), denetleme (güvenlik sorunlarını tespit, müşteri desteği ve uyumluluk için işlem izleme), güvenli süreçler arası iletişim (ideal olarak TLS ve kimlik doğrulama).
  - Monolitik uygulamada güvenlik (ekrandan, 00:31:00): istemci `POST /login` ile kullanıcı kimliği ve şifre gönderir; LoginHandler kimlik bilgilerini doğrular, kullanıcı bilgisini veritabanından alıp oturuma (session) yazar; istemciye `JSESSIONID` çerezi olarak oturum belirteci döner; sonraki isteklerde (`GET /orders/order-xyz`) çerez gönderilir; SessionBasedSecurityInterceptor oturumdan güvenlik bağlamını (security context) kurar; istek işleyici (OrderDetailsRequestHandler) bu bağlamı okuyarak işleme izin verip vermeyeceğine karar verir.
  - Bu yaklaşım mikroservislere doğrudan uygulanamaz: bellek içi güvenlik bağlamı paylaşılan bellek gerektirir; merkezi oturum ise servisleri ona bağımlı kılarak gevşek bağlılığı bozar.
  - Kimlik doğrulama API Gateway'de yapılmalıdır: servislerin kendisinin doğrulaması doğrulanmamış isteklerin iç ağa girmesine ve her serviste farklı mekanizmaların karmaşasına yol açar. Gateway güvenliği tek noktada toplar; doğrulama sonrası isteklere bir güvenlik belirteci (security token) ekler.
  - Yetkilendirme: gateway'de rol tabanlı URL erişimi uygulanabilir ama gateway'i servislere sıkı bağlar ve karmaşık kuralları (örneğin bir kullanıcının yalnız kendi siparişlerine erişmesi) desteklemez. Bunlar servislerin içinde ACL'lerle uygulanır.
  - Belirteç türleri (ekrandan, 00:37:05): opak belirteç dışarıdan bilgi içermez, doğrulama için güvenlik servisine eşzamanlı RPC çağrısı gerekir (ek gecikme, o servisin hatası tüm sistemi etkiler). Şeffaf belirteç kullanıcı bilgisini içerir; en yaygın format JWT'dir (payload ve imza). JWT'nin dezavantajı verilen belirtecin geri alınamamasıdır; çözüm kısa ömürlü belirteçlerdir, bu da sürekli yenileme gerektirir.
  - OAuth 2.0 (ekrandan, 00:39:15): yetkilendirme sunucusu (kimliği doğrular, access token ve refresh token verir), access token (kaynak sunucusuna erişim izni; genelde JWT), refresh token (uzun ömürlü, iptal edilebilir; yeni access token almak için), kaynak sunucusu (mikroservisler), istemci (API Gateway).
  - API istemcisinde akış (ekrandan, 00:40:00): istemci temel kimlik doğrulamayla istek gönderir (`GET /orders/1`, `Authorization: Basic ...`); gateway OAuth 2.0 sunucusuna Password Grant isteği yapar (`POST /oauth/token`); sunucu kimliği doğrulayıp `access_token` döner; gateway servis isteklerine `Authorization: Bearer AccessToken` ekler (Order Service). Oturum tabanlı istemcilerde gateway'in login handler'ı aynı yolla belirteci alıp istemciye döner, istemci sonraki isteklerine ekler.
- Dördüncü sunum: Microservice Chassis ve Service Mesh
  - Her mikroservis loglama, izleme (monitoring), sağlık kontrolü, güvenlik, hata yönetimi gibi kesişen kaygıları (cross-cutting concerns) karşılamak zorundadır. Bunları her serviste ayrı yazmak kod tekrarına ve kaynak israfına yol açar.
  - Microservice Chassis: bu kaygıları ele alan, servislerin hızlı ve yalın geliştirilmesini sağlayan çatı (iskelet). Klasik örnekler Spring Boot ve Spring Cloud; bir kütüphaneyle de aynı etki sağlanabilir. Servis yalnız kendi iş mantığına odaklanır.
  - Diyagram (ekrandan, 00:44:58): bütün servisler Chassis Skeleton'u kullanır (Service Runtime merkezde; Logging, Monitoring, Health Check, Service Discovery). Yalnız mesajlaşma ve veritabanı bağlantısı gereken servis ayrıca paylaşımlı kütüphanelere (Messaging, DB Connectivity) bağlanır.
  - Avantajlar: yazılacak kodu önemli ölçüde azaltır, yapısal sorunları ortadan kaldırarak iş mantığına odaklanmayı sağlar, kararlı ve yeniden kullanılabilir kod tabanı sunar; yeni özellik iskelete eklenir ve yalnız ona ihtiyaç duyan servisler etkilenir.
  - Dezavantaj: servislerin yazıldığı her dil/platform birleşimi için ayrı bir chassis gere-

kir.
  - Service Mesh: servise giren ve çıkan bütün ağ trafiğini yöneten ağ katmanı; devre kesici, dağıtık izleme, servis keşfi, yük dengeleme ve kural tabanlı trafik yönlendirme gibi işleri dilden bağımsız olarak servis dışında uygular. Trafik yönlendirme sayesinde dağıtım (deployment) sürümden (release) ayrılır: yeni sürüm üretime dağıtılır ama yalnız dahili test kullanıcıları gibi belli kullanıcılara açılır (ekrandan, 00:47:10).
  - Öğrencinin örnekleri: her isteğin loglanması için her serviste ayrı logger yazmak yerine ortak bir taban sınıftan kalıtım; mesh için SSL örneği, isteğin güvenli ve sertifikalı olup olmadığı alt katmanda denetlendikten sonra servis mantığına geçilmesi.

Hocanın Özellikle Vurguladığı Kısımlar

- Bu derste hoca akademik bir vurgu yapmadı; sunumları sırayla yönetti ve CQRS'in önceki hafta da işlendiğini hatırlattı. Sunumlarda öne çıkan noktalar aşağıdadır.
- CQRS view modülünde güncellemeler idempotent ve eşzamanlılığa dayanıklı olmalıdır
  - Tekrarlanan event'ler eşsiz event kimlikleriyle, eşzamanlı güncellemeler iyimser ya da kötümser kilitlemeyle ele alınır.
- Replikasyon gecikmesine karşı token
  - Güncelleme cevabında dönen event kimliği, sonraki sorguda görünümün güncel olup olmadığını anlamayı sağlar.
- End-to-end testler az sayıda yazılmalıdır
  - Yavaş, kırılgan ve pahalıdırlar; user journey testleri ile kapsam korunur, test sayısı azalır.
- Mikroservislerde kimlik doğrulama API Gateway'de, ince yetkilendirme servislerde yapılır
  - Gateway JWT gibi şeffaf belirteçlerle kullanıcı kimliğini ve rollerini servislere taşır; OAuth 2.0'da gateway istemci rolündedir.
- Chassis ile Mesh'in farkı
  - Chassis dile bağlı bir çatıdır, her dil için ayrısı gerekir; Mesh bu işleri dil bağımsız olarak ağ katmanında yapar.

Kısa Tekrar Notları

- CQRS view modülü: Query API + event handler'lar → Data Access (DAO) → view veritabanı.
- Gereksinimler: veritabanı/şema seçimi, idempotent ve eşzamanlılığa dayanıklı güncelleme, yeni view oluşturma mekanizması, replikasyon gecikmesine karşı önlem.
- Veritabanı seçimi sorgu türüne göre: PK araması → MongoDB/DynamoDB/Redis; metin araması → Elasticsearch; graf → Neo4j; BI raporlama → RDBMS.
- CQRS view için NoSQL genelde iyi seçimdir (basit işlemler); SQL de deneyim ve araç desteği nedeniyle kullanılabilir.
- Foreign key ile güncelleme: TeslimatAlındı event'inde sipariş kimliği yoksa teslimat kimliğiyle güncelleme.
- Eşzamanlı güncelleme → iyimser/kötümser kilitleme.
- İdempotentlik → eşsiz event kimliği; sıralı kimliklerde en büyük kimliği saklamak yeterli.
- Tutarlılık → güncelleme cevabında event kimliği içeren token.
- Yeni view: arşivlenmiş event'lerden (AWS S3, Apache Spark) ya da iki adımlı artımlı algoritmayla (snapshot + sonraki event'ler).
- End-to-end test: piramidin tepesi, yavaş, kırılgan, pahalı → az sayıda, user journey odaklı.
- Gherkin: Given/When/Then, iş paydaşlarının okuyabildiği DSL.
- Gradle Docker Compose eklentisi testlerden önce konteynerleri başlatır.
- Üretime hazırlık: güvenlik, yapılandırılabilirlik, gözlemlenebilirlik.
- Güvenliğin dört yönü: kimlik doğrulama, yetkilendirme, denetleme, güvenli iletişim (TLS).
- Monolit: oturum + bellek içi security context; mikroserviste uygulanamaz.
- Opak belirteç → RPC ile doğrulama; şeffaf belirteç (JWT) → servis kendi doğrular, ama geri alınamaz.
- OAuth 2.0 rolleri: authorization server, access token, refresh token, resource server, client (= API Gateway).

- Microservice Chassis: cross-cutting concerns için çatı (Spring Boot, Spring Cloud); her dil için ayrı chassis gerekir.
- Service Mesh: trafiği ağ katmanında yönetir; dağıtımı sürümden ayırır.

Detaylı Açıklamalar

- CQRS view modülünün iç yapısı
  - Önceki derste CQRS'in komut ve sorgu tarafını ayırdığı, sorgu tarafının event'lere abone olarak kendi görünümünü güncellediği anlatılmıştı. Bu derste o sorgu tarafının içi açılır. Görünüm modülünde iki tür istemci vardır: dışarıdan sorgu alan Query API ve komut tarafının event'lerini dinleyen event handler'lar. İkisi de veritabanına doğrudan değil, bir veri erişim modülü (DAO ve yardımcı sınıflar) üzerinden erişir. Böylece eşzamanlılık ve idempotentlik gibi zor sorunlar tek bir yerde çözülür.
- Eşzamanlılık ve idempotentlik neden ayrı sorunlardır?
  - Eşzamanlılık sorunu, farklı iki event'in aynı satırı aynı anda güncellemesidir; biri diğerinin değişikliğini ezerse veri kaybolur. Bu kilitleme ile çözülür. İdempotentlik sorunu ise aynı event'in tekrar gelmesidir. Slayttaki örnekte teslimat event'leri tekrarlandığı için görünüm bir süre PICKED_UP durumuna geri döner ve son DELIVERED event'i gelene kadar eskimiş veri gösterir. Bu geçici tutarsızlık kabul edilemiyorsa, handler işlediği event kimliklerini kaydeder ve tekrar geleni atlar.
- Replikasyon gecikmesi ve token
  - Görünüm event'lerle asenkron güncellendiği için bir güncellemenin sorgu tarafına yansıması zaman alır. İstemci güncellemeden hemen sonra sorgu yaparsa eski veriyi görebilir. Güncelleme komutunun cevabında event kimliği içeren bir token verilir. İstemci sorgusuna bu token'ı ekler; görünüm o event'i henüz işlemediyse hata döner, işlediyse güncel veri döner.
- Yeni view nasıl oluşturulur?
  - Yeni bir görünüm geçmiş event'lerden beslenmelidir. Mesaj kuyrukları event'leri kalıcı tutmadığı için (RabbitMQ işlenince siler, Kafka'nın tutma süresi sınırlıdır) event'lerin ayrıca arşivlenmesi gerekir. Ancak arşiv büyüdükçe her seferinde baştan işlemek pahalılaşır. Artımlı yöntemde her aggregate için periyodik snapshot alınır; view, en son snapshot ve ondan sonraki event'lerle kurulur. Bu bir checkpoint mantığıdır.
- End-to-end testleri az ama etkili tutmak
  - End-to-end testler sistemin bütününü doğruladıkları için değerlidir, ama her testte bütün servislerin ve altyapının ayağa kalkması gerekir. FTGO örneğinde yalnız konteynerlerin başlatılması ve testler birkaç dakika sürer. Bu yüzden sunumun önerisi, sipariş oluşturma, revize etme ve iptal gibi işlemleri tek bir user journey testinde birleştirmek, Gherkin gibi iş dilinde yazmak, testleri paralel çalıştırmak ve asıl yükü piramidin alt katmanlarına vermektir.
- Mikroservis güvenliği neden monolitten farklıdır?
  - Monolitte kullanıcı bir kez giriş yapar, oturum sunucunun belleğinde tutulur ve her istekte bellek içi güvenlik bağlamı kurulur. Mikroservislerde ortak bellek yoktur ve merkezi bir oturuma bağlanmak servisleri birbirine bağlar. Bu yüzden kimlik doğrulama API Gateway'e taşınır; gateway kullanıcının kimliğini ve rollerini bir belirteçle servislere taşır. Opak belirteç her istekte ek bir RPC çağrısı gerektirdiği için, servislerin kendi başına doğrulayabildiği JWT tercih edilir. OAuth 2.0 bu yapıyı standartlaştırır: gateway istemci, servisler kaynak sunucusu rolündedir.
- Chassis ve Mesh
  - Chassis, kesişen kaygıları servis koduna bir çatı olarak gömer; Java için Spring Boot/Spring Cloud gibi. Dezavantajı dil bağımlılığıdır. Service Mesh aynı kaygıların bir kısmını servis dışına, ağ katmanına taşır; böylece her dil için ayrı çatı gerekmez. Mesh'in trafik yönlendirmesi, yeni bir sürümü üretime dağıtıp önce yalnız test kullanıcılarına açmayı mümkün kılar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.